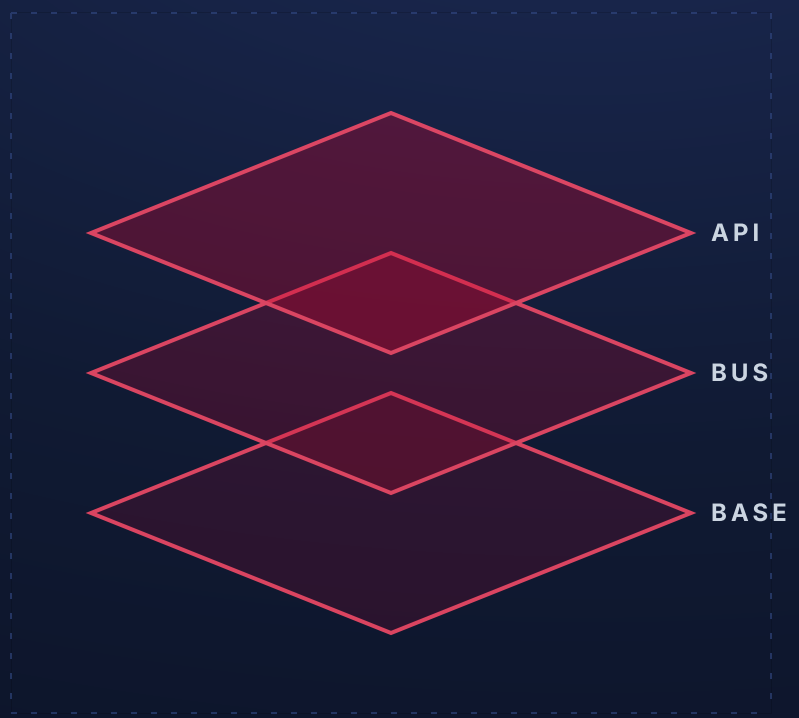
COMPRENDRE OPENSTACK · MODULE 3

Les composants système d'OpenStack

File de messages, base de données, serveur d'API : rôle, exposition, éléments de diagnostic

RabbitMQ | MariaDB + Galera | HAProxy + serveur WSGI | Commandes et journaux

Version de référence : OpenStack 2026.2 « Hibiscus » (30 septembre 2026)



INTRODUCTION

Objectif et déroulé

Objectif : connaître les trois logiciels sur lesquels reposent tous les services OpenStack, savoir ce que chacun expose sur le réseau, et savoir l'interroger quand le cloud se comporte mal.

À l'issue de ce module, vous saurez :

- Expliquer ce que font RabbitMQ, MariaDB Galera et le serveur d'API, et ce qu'ils ne font pas.
- Citer leurs **ports**, leurs comptes et les réseaux sur lesquels ils doivent — ou ne doivent pas — être joignables.
- Relier chaque option de configuration d'un service au composant qu'elle pilote.
- Lancer les **commandes de diagnostic** de chacun et interpréter leur sortie.
- Reconnaître dans un journal le message qui désigne le composant fautif.
- Conduire les reprises courantes : partition du bus, perte de quorum de la base, backend d'API hors service.

1 · La file de messages

RabbitMQ : anatomie, connexions, types de files, cluster, ports, comptes, options, commandes, alarmes, partitions, journaux, métriques.

2 · La base de données

MariaDB Galera : chaîne d'accès, réplication, quorum, ports, comptes, options, migrations, variables wsrep, erreurs, réamorçage, hygiène.

3 · Le serveur d'API

HAProxy et serveur WSGI : pile, workers, middlewares, sockets, surface exposée, délais, tests manuels, codes HTTP, journaux, saturation.

Méthode de diagnostic

Un arbre de décision commun aux trois composants, et une fiche réflexe.

INTRODUCTION

Trois composants sous tous les services



Conséquence pratique : savoir diagnostiquer ces trois composants, c'est savoir diagnostiquer la moitié des incidents de **n'importe quel** service — Nova, Neutron, Cinder ou Heat échouent de la même façon quand le bus ou la base ne répond plus.

INTRODUCTION

La grille de lecture : rôle, exposition, diagnostic

	File de messages	Base de données	Serveur d'API
Logiciel	RabbitMQ, en cluster de 3 nœuds	MariaDB + Galera, 3 nœuds	HAProxy + serveur WSGI (uWSGI ou Apache)
Rôle	Acheminer les appels RPC et les notifications	Conserver l'état durable de chaque service	Recevoir, authentifier et traiter les requêtes REST
Bibliothèque côté service	oslo.messaging	oslo.db (SQLAlchemy)	oslo.middleware, keystonemiddleware, paste
Port principal	5672 (TLS : 5671)	3306, à travers la VIP interne	Un port par API : 5000, 8774, 9696…
Qui s'y connecte	Les processus des services, depuis tous les nœuds	Les services des contrôleurs et des nœuds portant cinder-volume	Utilisateurs (VIP externe) et services (VIP interne)
Visible des utilisateurs ?	Jamais	Jamais	Oui : la surface publique du cloud, avec Horizon et les consoles
S'il s'arrête	Les opérations en cours restent figées	Toutes les API répondent en erreur	Le service concerné ne répond plus
Premier contrôle	`rabbitmqctl cluster_status`	`SHOW STATUS LIKE 'wsrep_%'`	`curl` sur la VIP, état des backends HAProxy

Rôle

Ce que le composant fait pour les services, et ce qu'il ne fait pas. Savoir ce qu'on peut en attendre évite de chercher au mauvais endroit.

Exposition

Ports, adresses d'écoute, comptes, chiffrement. Ce qui doit être joignable, par qui, et ce qui ne doit surtout pas l'être.

Diagnostic

Commandes, variables d'état, messages de journal et métriques qui permettent de dire « c'est lui » ou « ce n'est pas lui ».

INTRODUCTION

Dans le lab : ces composants avec Sunbeam

	Exemples de ce module (Kolla-Ansible)	Canonical OpenStack (Sunbeam)
Plan de contrôle	Conteneurs Docker ou Podman sur les contrôleurs	Pods de Canonical Kubernetes, gérés par des charms Juju (modèle `openstack`)
File de messages	RabbitMQ, un nœud par contrôleur	RabbitMQ, charm `rabbitmq-k8s`
Base de données	MariaDB + Galera, derrière HAProxy	MySQL InnoDB Cluster (`mysql-k8s`), joint par MySQL Router (`mysql-router-k8s`)
Entrée des API	HAProxy + Keepalived, deux VIP	Traefik (`traefik-k8s`), sur les adresses du répartiteur de Kubernetes
Bases OVN	Conteneurs sur les contrôleurs	`ovn-central-k8s`, `ovn-relay-k8s` ; MicroOVN sur les nœuds réseau
Hyperviseur	Conteneurs `nova_compute`, `nova_libvirt`, `ovn_controller`	Snap `openstack-hypervisor`
Lancer un outil	`docker exec rabbitmq rabbitmqctl …`	`sudo k8s kubectl exec -n openstack <pod> -c <conteneur> -- …`
Journaux	`/var/log/kolla/<service>/`	`sudo k8s kubectl logs -n openstack <pod> -c <conteneur>` · `juju debug-log -m openstack`
État d'ensemble	État des conteneurs sur chaque nœud	`juju status -m openstack` · `sunbeam cluster list`
Base : état, sauvegarde	`wsrep_%` · `kolla-ansible mariadb-backup`	Actions du charm MySQL : `get-cluster-status`, `create-backup`

Les commandes de ce module suivent Kolla-Ansible. Ce qui ne change pas avec Sunbeam : le rôle de chaque composant, les protocoles (REST, AMQP, SQL), les options oslo et leurs messages d'erreur, les commandes `openstack` et `rabbitmqctl`.

Ce qui change pour la base : réplication de groupe MySQL au lieu de Galera. Mêmes principes (majorité, nœud primaire unique), mais variables `group_replication_%` au lieu de `wsrep_%`.

01

La file de messages

RabbitMQ relie entre eux les processus d'un même service. Il est invisible tant qu'il fonctionne ; quand il faiblit, tout le cloud semble figé sans qu'aucune erreur n'apparaisse.

- Rôle et anatomie de RabbitMQ
- Cluster, ports, comptes et hôtes virtuels
- Commandes, lecture des files, alarmes
- Connexions, canaux, types de files
- Options côté services
- Partitions, journaux, métriques, interventions

Rôle : ce que le bus transporte, et pour qui

Trois services rendus

- **Appels RPC :** un processus demande à un autre processus du même service d'exécuter une fonction (créer une instance, attacher un volume).
- **Notifications :** un service annonce qu'un événement a eu lieu, pour qui voudra l'entendre (mesure d'usage, audit).
- **Découverte implicite :** un nova-compute devient joignable dès qu'il s'abonne à sa file. Il n'existe aucun registre d'adresses des processus.

Ce qu'il n'est pas

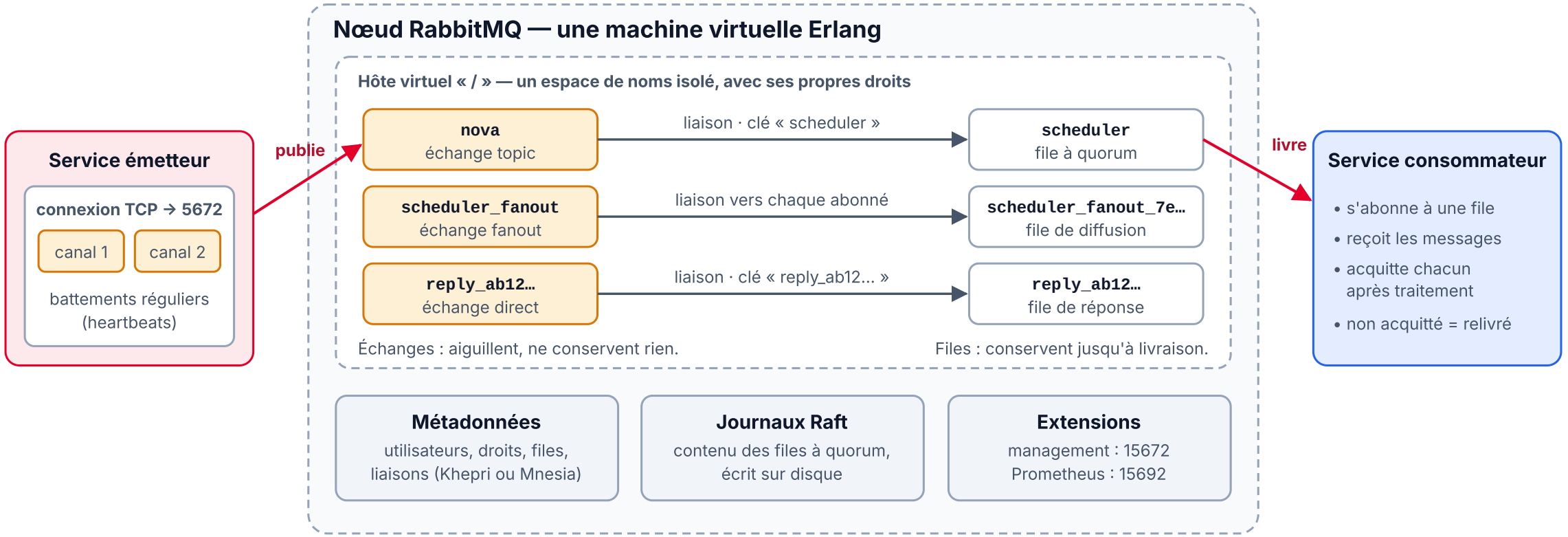
- Un stockage : un message consommé disparaît. L'état durable est en base.
- Un canal **entre** services : Nova parle à Neutron en REST, jamais par le bus.
- Un chemin de données : aucun paquet ni bloc d'instance n'y transite.

Découplage : l'émetteur ignore où tourne le destinataire et combien d'exemplaires existent. C'est ce qui permet d'ajouter un hyperviseur ou un scheduler sans rien reconfigurer.

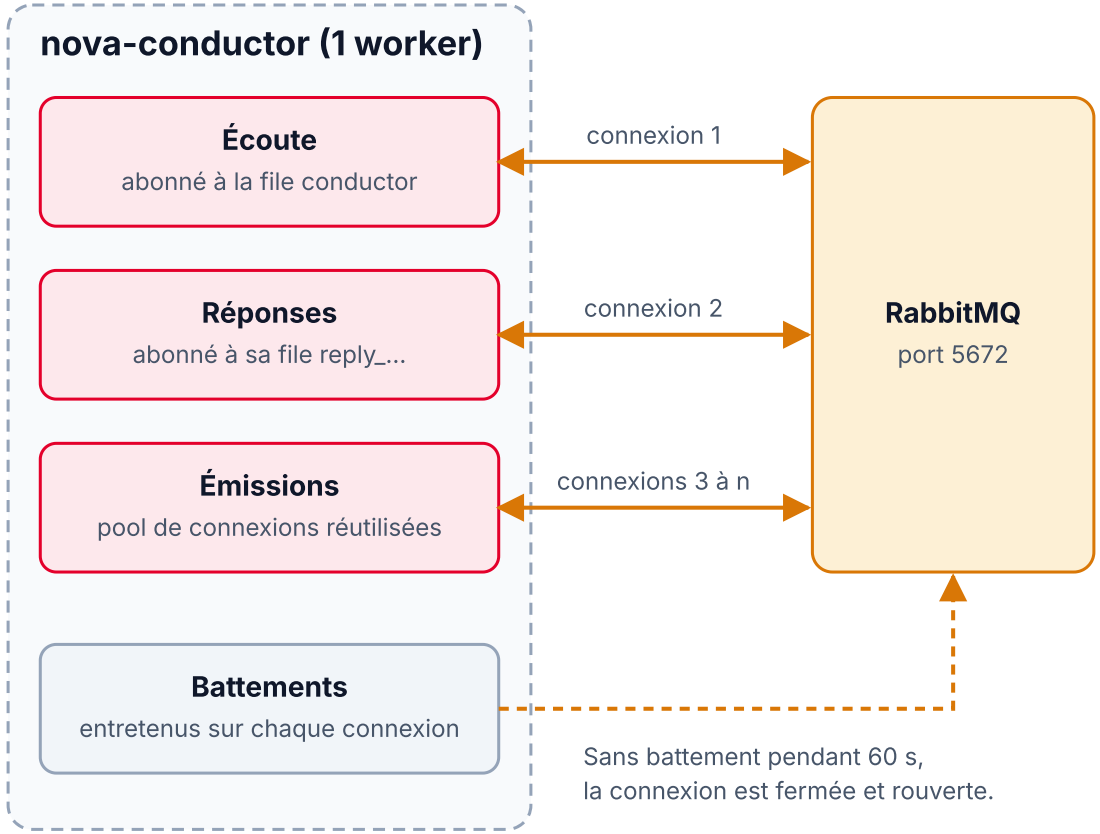
Service	RPC	Notifications	Remarque
Nova	●	●	Le plus gros utilisateur du bus
Cinder	●	●	
Neutron	○	●	RPC réduit au minimum avec OVN
Heat, Octavia, Magnum	●	●	API ↔ moteur ou contrôleur
Ironic, Manila, Designate	●	●	
Keystone	—	○	Notifications optionnelles (audit)
Glance	—	○	Notifications optionnelles
Placement	—	—	REST et SQL uniquement
Swift	—	—	N'utilise ni bus ni base SQL

● utilisé · ○ partiel ou optionnel. Tous partagent en général le même cluster RabbitMQ ; un cluster ou un hôte virtuel distinct par service est possible à grande échelle.

Anatomie d'un nœud RabbitMQ



Connexions et canaux : ce qu'un seul processus ouvre



- Une **connexion** est une session TCP authentifiée ; un **canal** est un flux logique à l'intérieur. oslo.messaging ouvre plusieurs connexions par processus.
- Chaque **worker** d'un service est un processus distinct, avec ses propres connexions.
- Le total croît donc vite : nombre de nœuds × services × workers × quelques connexions.

Option (oslo.messaging)	Défaut	Effet
`rpc_conn_pool_size`	30	Taille maximale du pool d'émission
`executor_thread_pool_size`	64	Messages traités en parallèle par processus
`heartbeat_timeout_threshold`	60 s	Silence toléré avant de couper

À vérifier côté RabbitMQ : la limite de descripteurs de fichiers du processus (une connexion = un descripteur). Un cluster qui refuse de nouvelles connexions alors que la mémoire est saine a souvent atteint cette limite.

Un pare-feu ou un répartiteur qui ferme les connexions inactives entre un nœud et RabbitMQ provoque des reconnexions en boucle : les battements servent précisément à garder ces sessions vivantes.

Les types de files et leur durée de vie

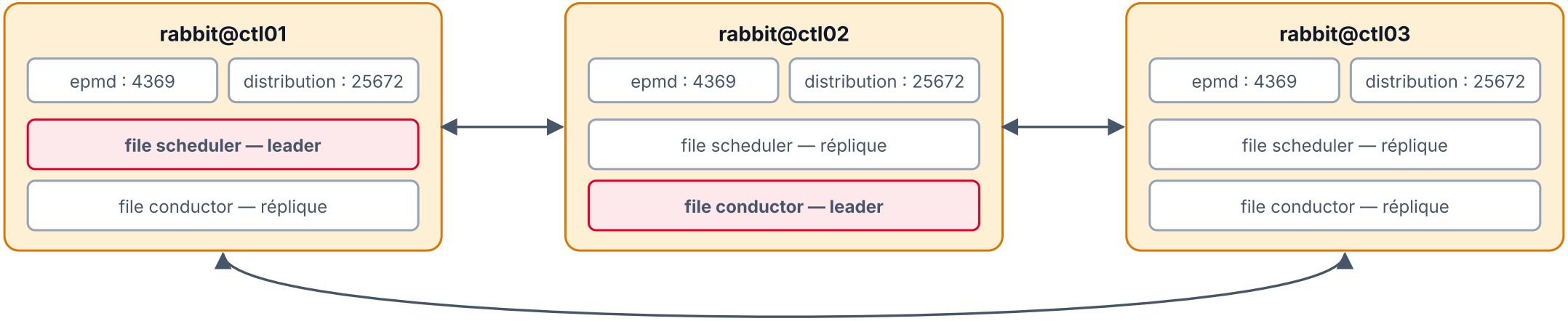
Type RabbitMQ	Réplication	Survit à la perte d'un nœud ?	Usage par oslo.messaging	Option
File classique	Aucune : vit sur un seul nœud	Non	Mode historique ; encore utilisé pour les files éphémères si rien n'est activé	—
File classique en miroir	Copies maître / miroirs	Oui, avec des cas de perte	Ancien mode haute disponibilité ; **supprimé** des versions récentes de RabbitMQ	politique `ha-mode`
File à quorum	Raft, majorité requise	Oui	Files RPC des services : `scheduler`, `compute.cmp01`…	`rabbit_quorum_queue`
File à quorum éphémère	Raft	Oui	Files de réponse et de diffusion	`rabbit_transient_quorum_queue`
Flux (stream)	Journal répliqué, lu par plusieurs abonnés	Oui	Diffusion (fanout) : un seul flux remplace une file par abonné	`rabbit_stream_fanout`

Durée de vie

- Les files RPC d'un rôle (`scheduler` , `conductor`) sont durables : elles attendent leurs consommateurs.
- Les files de réponse et de diffusion appartiennent à un processus : sans consommateur, elles expirent au bout de `rabbit_transient_queues_ttl` (1800 s par défaut).
- Un service arrêté longtemps laisse donc des ordres en attente dans sa file ciblée : ils seront exécutés à son retour.

Changer de type n'est pas transparent. Une file existe avec ses arguments ; un service qui la redéclare autrement est refusé : `PRECONDITION_FAILED - inequivalent arg 'x-queue-type'` . Le passage aux files à quorum impose d'arrêter les services, de supprimer les anciennes files, puis de redémarrer.

Le cluster : trois nœuds, un cookie, une majorité



Distribution Erlang entre les trois nœuds : même secret partagé (fichier .erlang.cookie), noms d'hôtes résolus de façon identique partout.

Réaction à une partition	Comportement	Usage
`pause_minority`	Le côté minoritaire se met en pause et refuse les clients	Recommandé à trois nœuds
`autoheal`	Les deux côtés continuent, puis le perdant est redémarré	Privilégie la disponibilité, au prix de pertes
`ignore`	Rien : les deux côtés divergent	À éviter

- Les **leaders** des files à quorum se répartissent sur les trois nœuds : aucun n'est « le maître ».
- Une file à quorum reste utilisable tant que **deux** de ses trois répliques se voient.
- Le cluster se forme par nom : un nœud dont le nom d'hôte change est vu comme un inconnu.
- Les services listent les trois nœuds et se reconnectent seuls : pas de VIP devant RabbitMQ.

Exposition : les ports de RabbitMQ

Port	Protocole	Usage	Qui doit l'atteindre	Réseau
5672	AMQP 0-9-1	Trafic des services, en clair	Tous les nœuds portant un service OpenStack	Gestion
5671	AMQP sur TLS	Même trafic, chiffré	Idem, à la place de 5672	Gestion
4369	epmd	Annuaire des nœuds Erlang	Les nœuds RabbitMQ entre eux, les outils en ligne de commande	Gestion, contrôleurs seulement
25672	Distribution Erlang	Réplication, élections, état du cluster	Les nœuds RabbitMQ entre eux	Gestion, contrôleurs seulement
15672	HTTP	Interface et API de gestion	Administrateurs, supervision	Gestion, accès restreint
15692	HTTP	Métriques au format Prometheus	Serveur de supervision	Gestion
35672–35682	Distribution Erlang	Outils `rabbitmqctl` lancés depuis un autre hôte	Postes d'administration, rarement	Gestion

Règle d'exposition : RabbitMQ n'écoute que sur l'adresse du réseau de gestion. Aucun de ces ports ne doit être joignable depuis le réseau public, ni depuis une instance. Qui peut publier sur le bus peut donner des ordres aux hyperviseurs.

Contrôle rapide : `rabbitmq-diagnostics listeners` affiche les ports réellement ouverts et l'interface associée ; `ss -ltnp | grep beam` le confirme côté système.

Exposition : comptes, hôtes virtuels et chaîne de connexion

Anatomie de l'option transport_url




```
# Créer un hôte virtuel, un compte et ses droits
rabbitmqctl add_vhost openstack
rabbitmqctl add_user nova 'S3cr3t'
rabbitmqctl set_permissions -p openstack nova ".*" ".*" ".*"
#                             configurer  écrire  lire

# Vérifier
rabbitmqctl list_users
rabbitmqctl list_permissions -p openstack
```

> Le compte `guest` par défaut n'est accepté que depuis l'hôte local ; il doit être supprimé ou désactivé en production.

Droit	Autorise
configurer	Créer et supprimer échanges et files (les services déclarent les leurs)
écrire	Publier dans un échange
lire	Consommer une file

```
[oslo_messaging_rabbit]     # chiffrement
ssl = true
ssl_ca_file = /etc/ssl/certs/ca-interne.pem
```

- Un hôte virtuel ou un compte par service limite l'effet d'un secret compromis.
- Le mot de passe figure en clair dans le fichier de configuration de chaque service : fichiers lisibles par le seul compte du service.

Les options côté services qui pilotent le bus

Option	Section	Défaut	Rôle	Quand y toucher
`transport_url`	[DEFAULT]	—	Adresse du bus pour le RPC	Toujours renseignée, avec les trois nœuds
`rpc_response_timeout`	[DEFAULT]	60 s	Attente maximale de la réponse à un « call »	Rarement : l'augmenter masque le vrai problème
`rabbit_quorum_queue`	[oslo_messaging_rabbit]	false	Déclare les files RPC en files à quorum	À activer partout, en une fois
`rabbit_transient_quorum_queue`	[oslo_messaging_rabbit]	false	Idem pour les files de réponse et de diffusion	Avec la précédente
`rabbit_stream_fanout`	[oslo_messaging_rabbit]	false	Diffusion par flux plutôt que par une file par abonné	Pour réduire le nombre de files
`use_queue_manager`	[oslo_messaging_rabbit]	false	Noms de files stables d'un redémarrage à l'autre	Évite l'accumulation de files orphelines
`heartbeat_timeout_threshold`	[oslo_messaging_rabbit]	60 s	Délai sans battement avant coupure	Si un équipement réseau coupe plus tôt
`driver`	[oslo_messaging_notifications]	(vide)	Émission des notifications : `messagingv2` ou `noop`	`noop` si personne ne les consomme

Cohérence obligatoire : tous les processus d'un même service — API, scheduler, agents sur chaque hyperviseur — doivent déclarer les files de la même façon. Une option changée sur les contrôleurs mais pas sur les nœuds de calcul casse le service.

Vérifier la valeur réellement lue par un processus : le fichier `/etc/<service>/<service>.conf` du conteneur ou de l'hôte, et le journal de démarrage en mode `debug`, qui liste toutes les options.

Diagnostic : les commandes à connaître

```
# 1. Le cluster est-il complet et sans alarme ?
rabbitmqctl cluster_status
rabbitmq-diagnostics check_running
rabbitmq-diagnostics alarms

# 2. Les files : messages en attente, consommateurs
rabbitmqctl list_queues name type messages_ready \
    messages_unacknowledged consumers

# 3. Seulement celles qui posent question
rabbitmqctl list_queues name messages consumers \
    | awk '$2 > 0 || $3 == 0'

# 4. Qui est connecté, depuis où, dans quel état
rabbitmqctl list_connections user peer_host state
rabbitmqctl list_consumers
```

Avec Kolla-Ansible, RabbitMQ tourne en conteneur : préfixer par `docker exec rabbitmq` (ou `podman exec`). Avec Sunbeam : `sudo k8s kubectl exec` dans le pod RabbitMQ (slide 5).

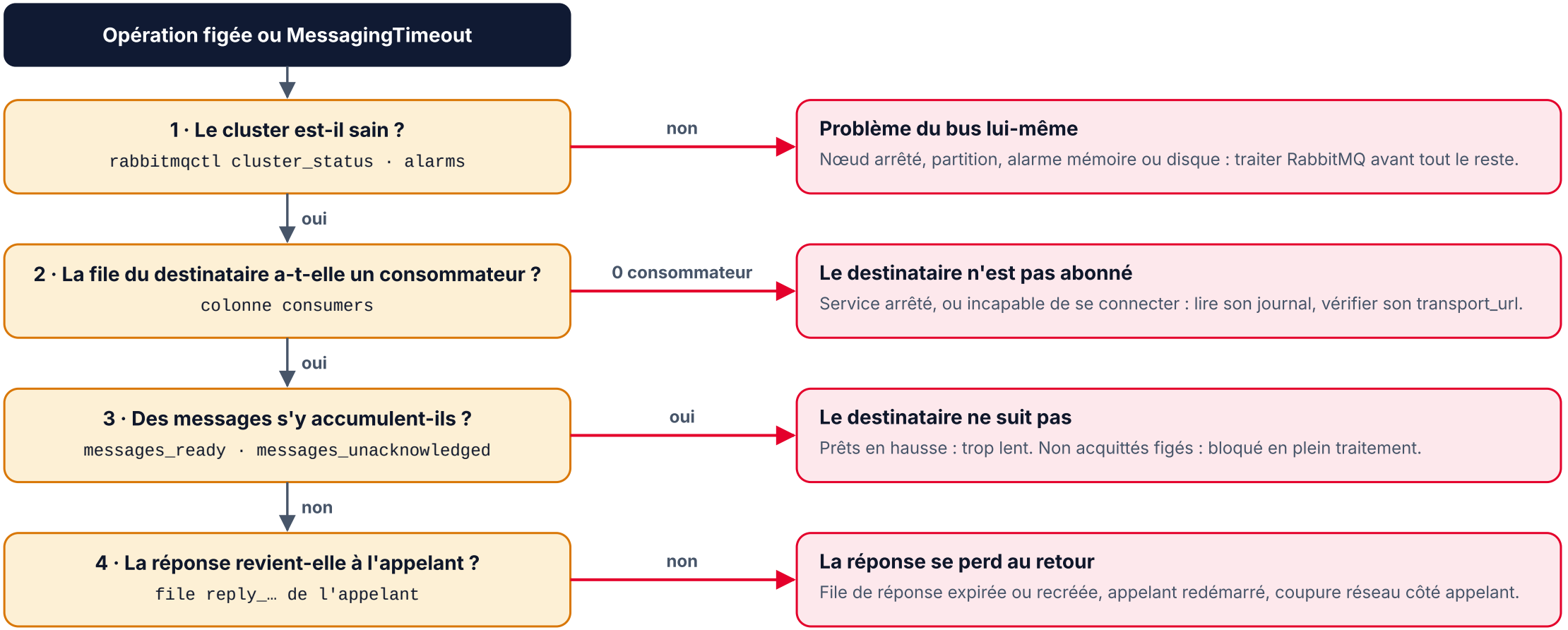
```
# 5. Ressources du nœud
rabbitmq-diagnostics memory_breakdown
rabbitmq-diagnostics status
rabbitmq-diagnostics listeners

# 6. Files à quorum : leader, membres, retard
rabbitmq-queues quorum_status scheduler
rabbitmq-queues check_if_node_is_quorum_critical
```

Ce que l'on cherche	Sortie saine
Nœuds en service	Trois « Running Nodes », aucune partition listée
Alarmes	Aucune alarme mémoire ni disque
Files RPC	0 message prêt, au moins un consommateur
Connexions	État `running` ; jamais `blocked`
Files à quorum	Trois membres en ligne, un leader

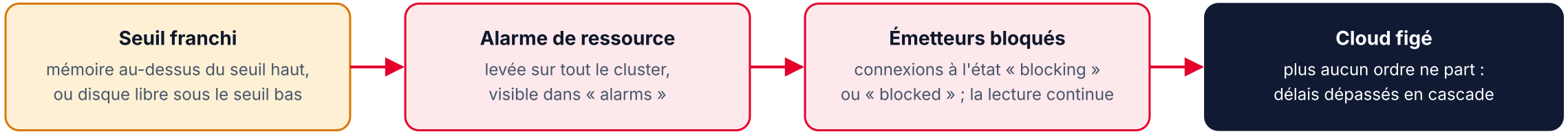
`check_if_node_is_quorum_critical` répond à la question à se poser **avant** d'arrêter un nœud : son arrêt ferait-il perdre le quorum à certaines files ?

Diagnostic : lire l'état des files pas à pas



Si les quatre réponses sont rassurantes, le bus est hors de cause : le traitement est simplement plus long que `rpc_response_timeout`, ou la panne se trouve derrière le destinataire (base, hyperviseur, stockage).

Alarmes de ressources : quand RabbitMQ bloque les émetteurs



Causes habituelles

- **Notifications sans consommateur** — une file qui grossit sans fin
- **Destinataires arrêtés** — leurs files ciblées se remplissent
- **Files et connexions orphelines** — redémarrages en boucle d'un service

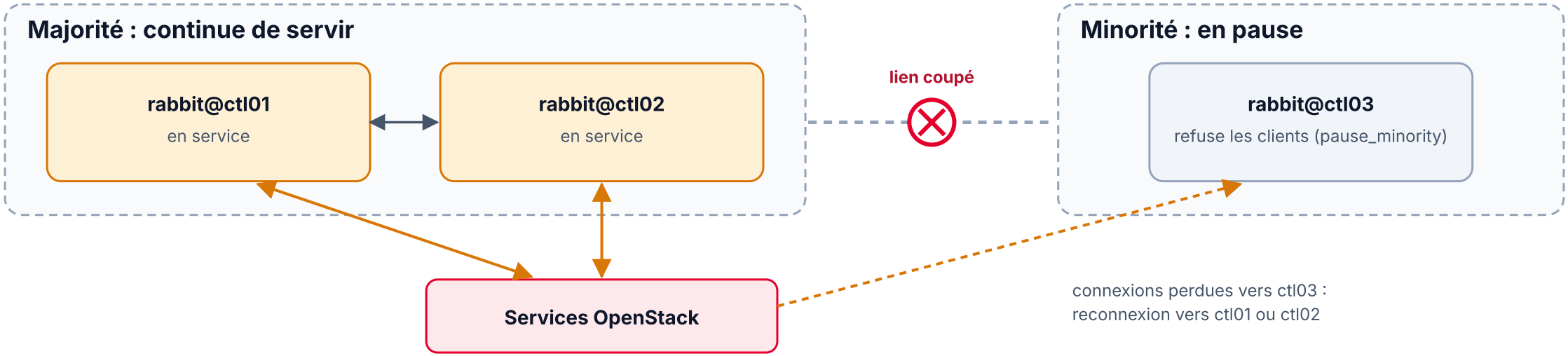
Paramètre RabbitMQ	Effet
`vm_memory_high_watermark`	Part de la mémoire au-delà de laquelle l'alarme se lève (0,4 à 0,6 selon la version)
`disk_free_limit`	Espace disque minimal ; la valeur par défaut, très basse, est à relever

```
rabbitmq-diagnostics alarms
rabbitmqctl list_connections name state | grep -i block
rabbitmq-diagnostics memory_breakdown
```

- Le blocage est un **mécanisme de protection** : RabbitMQ préfère ralentir les émetteurs plutôt que tomber faute de mémoire.
- Vu des services, rien n'échoue tout de suite : les envois attendent, puis les appelants dépassent leur délai.
- Dans un conteneur, le seuil se calcule sur la mémoire de l'hôte si la limite du conteneur n'est pas déclarée à RabbitMQ.

Sortie de crise : trouver la file qui enfle (`list_queues` triée par taille), rétablir son consommateur ou, s'il s'agit de notifications inutiles, la purger et passer le pilote à `noop` .

Partition réseau : symptômes et reprise



Ce que l'on observe

- Dans les journaux des services : vagues de « AMQP server … is unreachable », puis reconnexions.
- `cluster_status` : section « Network Partitions » non vide, ou un nœud absent des « Running Nodes ».

Reprise après retour du lien

- Avec `pause_minority`, le nœud isolé rejoint seul le cluster.
- S'il reste à l'écart : `rabbitmqctl stop_app` puis `start_app` sur ce nœud.

Arrêt complet du cluster

- Le **dernier nœud arrêté** doit redémarrer en premier : lui seul connaît le dernier état.
- Si cet ordre est inconnu ou impossible : `rabbitmqctl force_boot` sur un nœud, puis démarrage des autres.

Après toute reprise, contrôler les services eux-mêmes : certains processus restent connectés à une file qui n'existe plus et ne s'en aperçoivent pas. Vérifier la colonne `consumers`, redémarrer ceux qui manquent.

Journaux : reconnaître les messages liés au bus

Message	Où	Ce qu'il signifie	Suite à donner
`AMQP server on 10.20.0.11:5672 is unreachable … Trying again in 1 seconds`	Journal du service	Connexion impossible ou rompue vers ce nœud	Normal lors d'un redémarrage ; anormal s'il se répète sur les trois nœuds
`MessagingTimeout: Timed out waiting for a reply to message ID …`	Journal de l'appelant	Aucun destinataire n'a répondu dans le délai	Arbre de diagnostic des files
`missed heartbeats from client, timeout: 60s`	Journal RabbitMQ	Un client n'a plus donné signe de vie : processus gelé ou réseau coupé	Chercher l'hôte en cause ; charge, pare-feu, MTU
`Connection reset by peer` , `Broken pipe`	Journal du service	Session TCP fermée par l'autre extrémité ou par un équipement intermédiaire	Sans gravité si isolé ; sinon délai d'inactivité d'un pare-feu
`NOT_FOUND - no queue 'reply_…' in vhost '/'`	Journal du service	La file de réponse de l'appelant a disparu	Redémarrer l'appelant ; fréquent après une réinitialisation du bus
`PRECONDITION_FAILED - inequivalent arg 'x-queue-type'`	Journal du service	File déjà déclarée avec un autre type	Supprimer l'ancienne file ; aligner l'option sur tous les nœuds
`ACCESS_REFUSED - Login was refused`	Les deux	Compte ou mot de passe incorrect, hôte virtuel inexistant	Comparer `transport_url` et `list_users`
`memory resource limit alarm set on node …`	Journal RabbitMQ	Alarme mémoire : les émetteurs sont bloqués	Trouver la file qui enfle

Où lire : RabbitMQ écrit dans `/var/log/rabbitmq/rabbit@<hôte>.log` (`/var/log/kolla/rabbitmq/` avec Kolla). Les services écrivent dans leur propre journal, sous le nom de module `oslo.messaging._drivers…`

Lecture croisée : un même incident laisse une trace des deux côtés. L'heure exacte de la coupure dans le journal RabbitMQ permet de retrouver, par horodatage, tous les services touchés.

Métriques à surveiller

Métrique	Valeur saine	Signal d'alerte	Ce que cela révèle
Nœuds en service	3 sur 3	Moins de 3	Redondance perdue ; à 1 sur 3, les files à quorum s'arrêtent
Partitions	Aucune	Toute partition	Problème réseau entre contrôleurs
Messages prêts, par file	Proche de 0	Croissance continue	Consommateur absent ou trop lent
Messages non acquittés	Faible, fluctuant	Valeur figée	Consommateur bloqué pendant un traitement
Consommateurs par file RPC	≥ 1	0	Service arrêté ou déconnecté
Mémoire utilisée / seuil	Nettement sous le seuil	Approche du seuil	Alarme imminente : émetteurs bientôt bloqués
Espace disque libre	Large marge	Approche de la limite	Idem ; journaux Raft des files à quorum
Connexions et canaux	Stable	Hausse continue ou dents de scie	Fuite de connexions, service qui redémarre en boucle
Descripteurs de fichiers	Loin de la limite	Proche de la limite	Nouvelles connexions bientôt refusées
Nombre de files	Stable	Hausse continue	Files éphémères qui s'accumulent

Sources : extension Prometheus (port 15692) pour la collecte continue ; interface de gestion (port 15672) pour l'exploration ; `rabbitmqctl` pour le contrôle ponctuel.

L'alerte la plus utile : « une file RPC a plus de N messages prêts depuis plus de quelques minutes ». Elle précède presque tous les incidents visibles par les utilisateurs.

Intervenir sur le bus sans aggraver la situation

Intervention	Effet sur le cloud	Précaution
Redémarrer un nœud RabbitMQ	Reconnexion des clients vers les deux autres ; quelques appels en cours échouent	Un nœud à la fois ; vérifier d'abord `check_if_node_is_quorum_critical`
Redémarrer un service OpenStack	Ses files éphémères sont recréées ; ses ordres en attente sont repris	Sans risque pour le bus
Purger une file de notifications	Mémoire libérée ; événements perdus	Acceptable si personne ne les consomme ; corriger la configuration ensuite
Purger une file RPC	Les ordres en attente sont perdus : ressources figées dans un état transitoire	À éviter ; prévoir de remettre à plat l'état des ressources concernées
Supprimer une file	Le service abonné ne reçoit plus rien jusqu'à son redémarrage	Uniquement service arrêté, pour un changement de type
Réinitialiser le cluster	Tout est perdu : comptes, droits, files, messages	Dernier recours ; recréer les comptes puis redémarrer tous les services
Monter RabbitMQ de version	Nœud par nœud, sans coupure si les files sont à quorum	Activer les « feature flags » demandés avant de passer à la version suivante

Ce qu'un message perdu laisse derrière lui : une instance en BUILD, un volume en « attaching », une migration à mi-chemin. Le bus ne rejoue rien de lui-même ; il faut remettre l'état en cohérence avec les commandes d'administration du service.

Bonne pratique : avant une opération de maintenance sur le bus, s'assurer qu'aucune opération longue n'est en cours — migrations, créations en masse, sauvegardes de volumes.

02

La base de données

MariaDB conserve tout ce que le cloud sait de lui-même. Galera en maintient trois copies identiques. Sans elle, aucune API ne peut répondre — mais les instances, elles, continuent de tourner.

- Rôle et contenu
- Quorum et états des nœuds
- Migrations de schéma
- Chaîne d'accès et réplication Galera
- Ports, comptes, options côté services
- Variables wsrep, erreurs, réamorçage, hygiène

Rôle : la mémoire du cloud

Ce qu'elle contient

- L'**inventaire** : instances, ports, réseaux, volumes, images, utilisateurs, projets.
- L'**état** de chaque ressource et de chaque opération en cours (`vm_state`, `task_state`, `status`).
- Les **affectations** : quelle instance sur quel hôte, quel volume attaché où, quelles allocations dans Placement.
- Les **battements** des services et la configuration dynamique : quotas, gabarits, agrégats, rôles.

Ce qu'elle ne contient pas

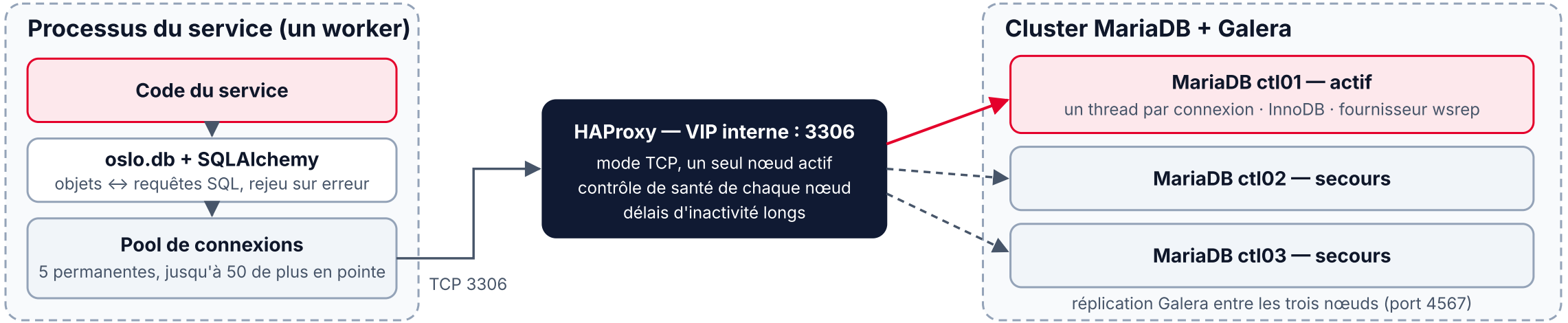
- Les données des utilisateurs : disques, objets et images vivent dans le stockage.
- Les jetons Keystone : au format Fernet, ils ne sont pas enregistrés.
- L'état réel des hyperviseurs : la base dit ce qui **devrait** être, pas ce qui est.

La base est la **référence**. En cas de divergence avec la réalité (instance disparue de l'hyperviseur, port orphelin), ce sont les services qui se resynchronisent sur elle — ou l'administrateur qui la corrige par les API, jamais par SQL.

Schéma	Tables caractéristiques	Croissance
`keystone`	`project`, `user`, `assignment`, `endpoint`	Faible
`nova_api`	`flavors`, `request_specs`, `cell_mappings`, `quotas`	Modérée
`nova` (cellule)	`instances`, `instance_actions`, `block_device_mapping`, `services`	Forte
`nova_cell0`	Instances jamais placées sur un hôte	Faible
`placement`	`resource_providers`, `inventories`, `allocations`	Modérée
`neutron`	`networks`, `ports`, `ipallocations`, `securitygrouprules`	Modérée
`cinder`	`volumes`, `snapshots`, `volume_attachment`	Modérée
`glance`	`images`, `image_locations`, `image_properties`	Faible
`heat`	`stack`, `resource`, `event`	Forte

« Forte » : des lignes s'ajoutent à chaque opération et les suppressions ne font que les marquer. Sans purge, ces schémas grossissent sans limite.

La chaîne d'accès : du service à la ligne écrite

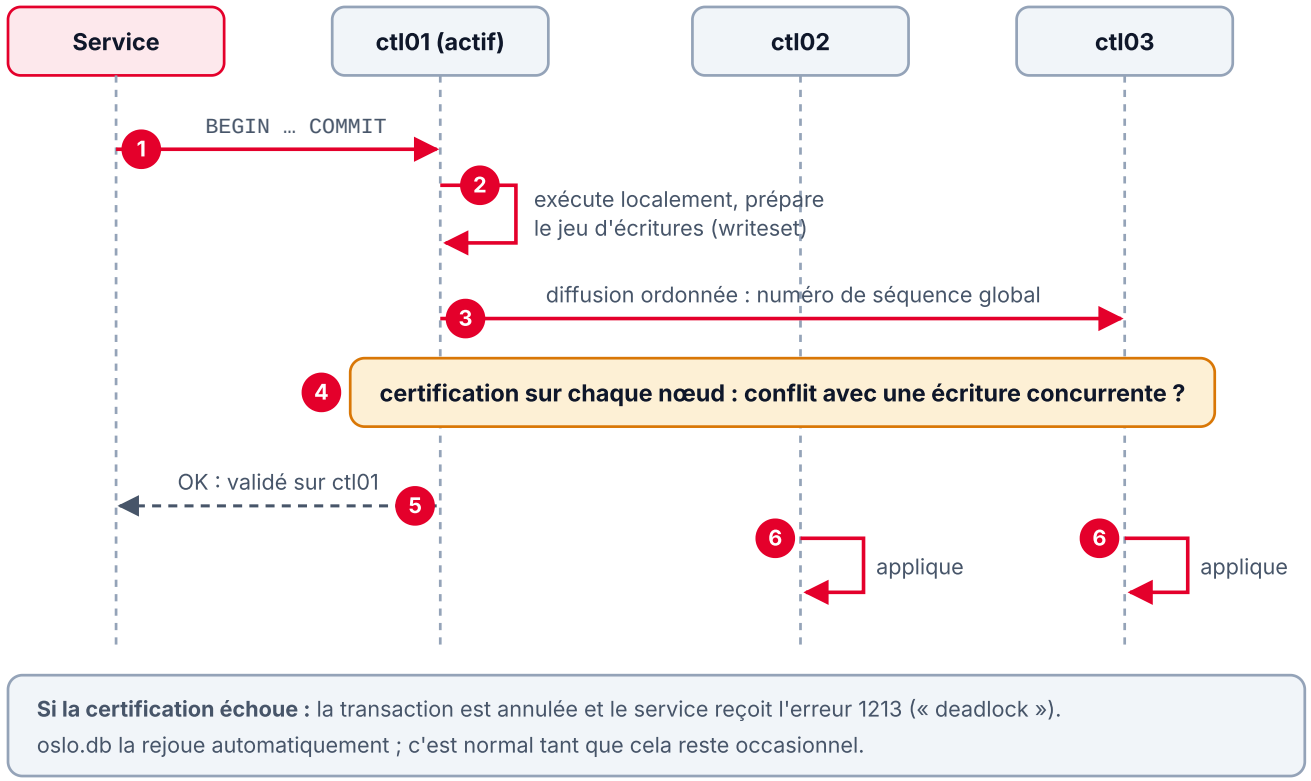


Délai d'inactivité	Réglé où	Ordre de grandeur
`connection_recycle_time`	Service, section `[database]`	600 s (défaut 3600 : à abaisser)
`timeout client` / `server`	HAProxy, bloc MariaDB	3600 s
`wait_timeout`	MariaDB	28800 s

Règle : recyclage côté service < délai HAProxy < délai MariaDB. Sinon un équipement ferme une connexion que le pool croit vivante : « MySQL server has gone away ».

- Les services ne connaissent que la **VIP** : ils ignorent quel nœud est actif.
- HAProxy envoie tout vers **un seul nœud** ; les deux autres ne servent qu'en cas de panne du premier.
- Une bascule coupe toutes les connexions ouvertes : oslo.db les rouvre et rejoue la requête en cours.
- Chaque connexion occupe un thread et de la mémoire côté MariaDB : la somme de tous les pools doit rester sous `max_connections`.

Galera : la réplication par certification



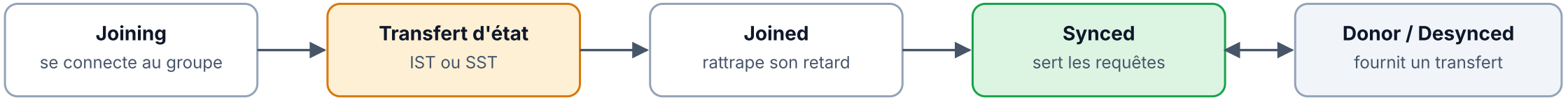
Si la certification échoue : la transaction est annulée et le service reçoit l'erreur 1213 (« deadlock »).
oslo.db la rejoue automatiquement ; c'est normal tant que cela reste occasionnel.

- **Synchrone à la validation :** quand le service reçoit « OK », la transaction est connue et acceptée par la majorité. Aucune écriture validée ne se perd à la panne d'un nœud.
- **Appliquée ensuite :** les autres nœuds l'écrivent avec un léger décalage. Lire sur un autre nœud juste après peut renvoyer l'ancienne valeur.
- **Tous les nœuds acceptent les écritures**, mais deux transactions concurrentes sur les mêmes lignes via deux nœuds se heurtent à la certification. D'où le nœud d'écriture unique.
- **Contrôle de flux :** un nœud trop lent à appliquer freine tout le cluster. Les trois nœuds doivent avoir des disques équivalents.

Contraintes de Galera : tables InnoDB uniquement, clé primaire sur chaque table, grosses transactions à éviter. Les schémas d'OpenStack les respectent.

Quorum et états d'un nœud Galera

Composant primaire (Primary) : le groupe de nœuds qui détient la majorité — le seul autorisé à traiter des requêtes



Hors composant primaire (non-Primary) : nœud isolé ou groupe minoritaire — refuse toute requête (erreur 1047)

Transfert	Principe	Quand	Durée
IST (incrémental)	Rejoue les transactions manquantes depuis le cache d'un donneur	Absence courte : le retard tient dans le cache (`gcache`)	Secondes à minutes
SST (complet)	Copie intégrale des données depuis un donneur (mariabackup)	Nœud neuf, absence longue, données corrompues	Proportionnelle à la taille de la base

- **Majorité stricte :** à trois nœuds, deux doivent se voir. À deux, la moindre coupure entre eux arrête tout.
- **Nombre pair :** ajouter un arbitre (`garbd`) qui vote sans stocker de données.
- **Arrêt propre** d'un nœud : il quitte le groupe et la majorité se recalcule sur les nœuds restants.
- **Arrêt brutal** des trois : plus de composant primaire, aucun nœud ne redémarre seul en service — il faut réamorcer le cluster.
- Pendant un SST, le donneur est mis à l'écart : à trois nœuds, un seul reste pleinement disponible.

Exposition : ports, écoute et répartiteur

Port	Usage	Qui doit l'atteindre
3306	Connexions SQL des clients	HAProxy des contrôleurs ; services, via la VIP
4567	Réplication Galera (TCP, et UDP en multidiffusion)	Les nœuds MariaDB entre eux
4568	Transfert incrémental (IST)	Les nœuds MariaDB entre eux
4444	Transfert complet (SST)	Les nœuds MariaDB entre eux
Port du contrôle de santé	Script qui répond « nœud synchronisé ou non »	HAProxy uniquement

- MariaDB écoute sur l'adresse du **réseau de gestion** de son nœud (`bind-address`), jamais sur toutes les interfaces.
- La VIP interne porte le même port 3306, sur une autre adresse : HAProxy et MariaDB cohabitent sur le même hôte.
- Aucun de ces ports n'est ouvert vers les nœuds de calcul, les instances ou le réseau public.

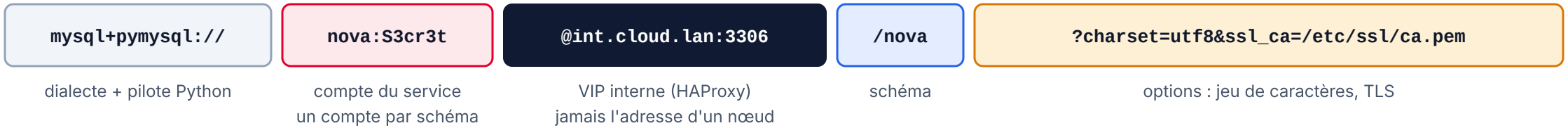
Contrôle : `ss -ltnp | grep -E '3306|4567'` sur chaque contrôleur ; depuis un nœud de calcul, `nc -zv <VIP> 3306` doit **échouer**.

```
# haproxy.cfg — bloc MariaDB (forme type)
listen mariadb
    mode tcp
    bind 10.20.0.10:3306
    option tcpka
    timeout client 3600s
    timeout server 3600s
    option httpchk
    server ctl01 10.20.0.11:3306 check port 4569 inter 2000
    server ctl02 10.20.0.12:3306 check port 4569 … backup
    server ctl03 10.20.0.13:3306 check port 4569 … backup
```

- `mode tcp` : HAProxy relaie la session sans la comprendre.
- `backup` : ces serveurs ne reçoivent du trafic que si le premier est déclaré hors service.
- `check port` : HAProxy interroge un petit service local à chaque nœud, qui ne répond « 200 » que si `wsrep_local_state` vaut *Synced*. Le numéro de port dépend de l'outil de déploiement.
- Sans ce contrôle, HAProxy enverrait des requêtes à un nœud vivant mais hors quorum.

Exposition : comptes, privilèges et chaîne de connexion

Anatomie de l'option connection




```
-- Ce que l'outil de déploiement crée pour chaque service
CREATE DATABASE nova;
CREATE USER 'nova'@'10.20.0.%' IDENTIFIED BY 'S3cr3t';
GRANT ALL PRIVILEGES ON nova.* TO 'nova'@'10.20.0.%';

-- Vérifier
SELECT user, host FROM mysql.user;
SHOW GRANTS FOR 'nova'@'10.20.0.%';
```

> Avec HAProxy en mode TCP, MariaDB voit l'adresse du **répartiteur**, pas celle du service : la restriction par hôte porte donc sur les contrôleurs.

Compte	Portée	Utilisé par
Un par service	Tous les droits sur son seul schéma	Les processus du service
Contrôle de santé	Connexion sans privilège	Script interrogé par HAProxy
Transfert d'état	Lecture complète, rechargement	mariabackup pendant un SST
Supervision	Lecture des variables d'état	Exporteur de métriques
root	Tout	Administrateur, en local seulement

- Un service compromis ne lit que son schéma : Glance n'accède pas aux tables de Keystone.

Les options côté services qui pilotent l'accès à la base

Option de la section [database]	Défaut	Rôle	Point d'attention
`connection`	—	Adresse, compte et schéma	Toujours la VIP interne ; contient un mot de passe en clair
`max_pool_size`	5	Connexions gardées ouvertes par processus	Multiplié par le nombre de workers et de contrôleurs
`max_overflow`	50	Connexions supplémentaires en pointe	Borne le pire cas : 55 connexions par processus
`pool_timeout`	30 s	Attente d'une connexion libre dans le pool	Dépassement = service saturé, pas base en panne
`connection_recycle_time`	3600 s	Âge maximal d'une connexion avant renouvellement	Défaut égal au délai HAProxy usuel : l'abaisser (600 s, par exemple)
`max_retries` · `retry_interval`	10 · 10 s	Tentatives de connexion au démarrage	Un service démarré avant la base patiente, puis abandonne
`db_max_retries`	20	Rejeux d'une transaction après erreur de connexion ou « deadlock »	Absorbe les conflits de certification occasionnels
`slave_connection`	(vide)	Seconde adresse pour certaines lectures	Rarement utilisé avec Galera
`connection_debug`	0	Journalise les requêtes SQL (0 à 100)	Très verbeux : diagnostic ponctuel uniquement

Nova a plusieurs bases : `[api_database]` pour le schéma `nova_api`, `[database]` pour la cellule. Les nœuds de calcul n'ont ni l'une ni l'autre section renseignée.

Côté MariaDB, les réglages qui répondent à ceux-ci : `max_connections`, `wait_timeout`, `innodb_buffer_pool_size` (la mémoire consacrée aux données) et `wsrep_provider_options` (dont `gcache.size`).

Schémas et migrations : la base évolue avec le code

Chaque service possède son schéma et l'outil qui le fait évoluer. À chaque montée de version, le schéma doit être migré **avant** que le nouveau code ne s'en serve.

Service	Créer ou migrer le schéma	Migrations de données en ligne	Version courante
Keystone	`keystone-manage db_sync` (phases `--expand`, `--migrate`, `--contract`)	—	`keystone-manage db_version`
Nova	`nova-manage api_db sync` puis `nova-manage db sync`	`nova-manage db online_data_migrations`	`nova-manage db version`
Placement	`placement-manage db sync`	`placement-manage db online_data_migrations`	`placement-manage db version`
Neutron	`neutron-db-manage upgrade heads` (ou `--expand` puis `--contract`)	—	`neutron-db-manage current`
Cinder	`cinder-manage db sync`	`cinder-manage db online_data_migrations`	`cinder-manage db version`
Glance	`glance-manage db sync` (ou `expand`, `migrate`, `contract`)	—	`glance-manage db version`
Heat	`heat-manage db_sync`	—	`heat-manage db_version`

Étendre, puis contracter

Phase 1 : ajouter colonnes et tables, sans rien retirer — l'ancien code fonctionne encore. Phase 2 : supprimer l'ancien, une fois tout le code mis à jour. C'est ce qui permet une montée de version sans arrêt des API.

Migrations en ligne

Certaines transformations de données se font par lots, services en marche. Elles doivent être **terminées** avant la montée de version suivante : c'est un contrôle de `nova-status upgrade check`.

Symptôme d'un oubli

Erreurs « Unknown column », « Table doesn't exist » ou refus de démarrer du service juste après une mise à jour : le schéma est en retard sur le code. Jamais de modification manuelle du schéma.

Diagnostic : l'état du cluster Galera

```
MariaDB> SHOW GLOBAL STATUS LIKE 'wsrep_%';
+----------------------------+--------------+
| wsrep_cluster_size         | 3            |
| wsrep_cluster_status       | Primary      |
| wsrep_connected            | ON           |
| wsrep_ready                | ON           |
| wsrep_local_state_comment  | Synced       |
| wsrep_incoming_addresses   | 10.20.0.11…  |
| wsrep_last_committed       | 48211937     |
| wsrep_local_recv_queue_avg | 0.02         |
| wsrep_flow_control_paused  | 0.00         |
| wsrep_local_cert_failures  | 117          |
+----------------------------+--------------+
```

```
# En une ligne, sur chaque contrôleur
mariadb -e "SHOW STATUS WHERE Variable_name IN
  ('wsrep_cluster_size','wsrep_cluster_status',
   'wsrep_local_state_comment','wsrep_ready')"
```

Variable	Sain	Sinon
`wsrep_cluster_size`	3	Un nœud manque : redondance réduite
`wsrep_cluster_status`	Primary	Ce nœud est hors quorum et refuse les requêtes
`wsrep_ready`	ON	Le nœud n'accepte pas de requêtes
`wsrep_local_state_comment`	Synced	Joining, Donor… : nœud en transfert
`wsrep_last_committed`	Identique sur les trois	Écart durable : un nœud décroche
`wsrep_local_recv_queue_avg`	Proche de 0	Ce nœud applique trop lentement
`wsrep_flow_control_paused`	Proche de 0	Part du temps où le cluster a été freiné
`wsrep_local_cert_failures`	Croissance lente	Hausse rapide : écritures sur plusieurs nœuds

Interroger les trois nœuds, pas seulement la VIP : chacun donne sa propre vision. Trois nœuds qui annoncent chacun `cluster_size = 1` forment trois clusters distincts — la pire situation.

Diagnostic : connexions, requêtes et volume

```
-- Connexions : utilisées, record, plafond
SHOW GLOBAL STATUS LIKE 'Threads_connected';
SHOW GLOBAL STATUS LIKE 'Max_used_connections';
SHOW VARIABLES LIKE 'max_connections';

-- Qui est connecté, par compte
SELECT user, COUNT(*) FROM information_schema.processlist
  GROUP BY user ORDER BY 2 DESC;

-- Requêtes en cours depuis plus de 5 s
SELECT id, user, time, state, LEFT(info, 80)
  FROM information_schema.processlist
  WHERE command <> 'Sleep' AND time > 5;

-- Verrous et dernier interblocage
SHOW ENGINE INNODB STATUS\G
```

```
-- Taille de chaque schéma, en Mo
SELECT table_schema,
       ROUND(SUM(data_length + index_length)/1048576) AS mo
  FROM information_schema.tables
  GROUP BY table_schema ORDER BY mo DESC;
```

- **Threads_connected proche de max_connections :** la somme des pools dépasse ce que la base accepte. Réduire les workers ou relever le plafond, mémoire à l'appui.
- **Un compte domine la liste :** le service correspondant fuit des connexions ou a trop de workers.
- **Requêtes longues :** souvent des listes non paginées sur des tables jamais purgées (`instances` , `instance_actions` , `event`).
- **Schéma démesuré :** lignes marquées supprimées qui s'accumulent — voir l'archivage.

> Le **journal des requêtes lentes** (`slow_query_log` , `long_query_time`) s'active à chaud et donne, en quelques heures, la liste des requêtes à traiter.

Journaux : reconnaître les erreurs liées à la base

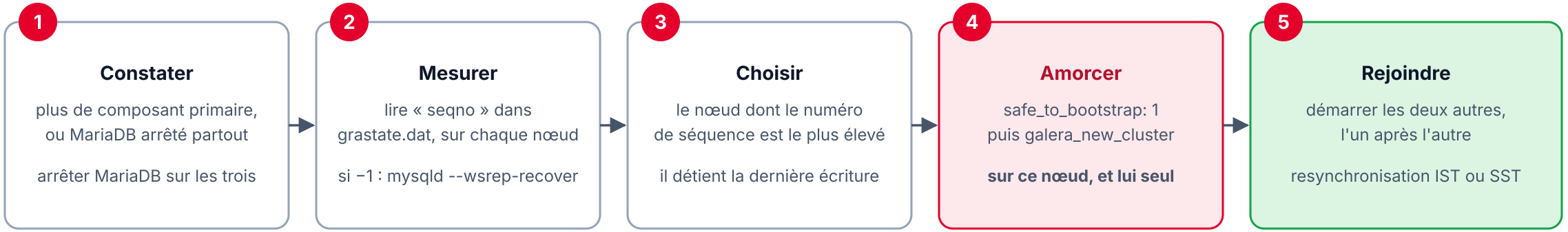
Message dans le journal d'un service	Code	Signification	Cause probable
`DBConnectionError … Can't connect to MySQL server`	2003	Connexion refusée ou sans réponse	VIP interne absente, HAProxy sans nœud sain, pare-feu
`Lost connection to MySQL server during query`	2013	Session coupée en cours de requête	Bascule de nœud, redémarrage, délai d'un équipement
`MySQL server has gone away`	2006	Le pool a réutilisé une connexion déjà fermée	Délais d'inactivité mal ordonnés entre service, HAProxy et MariaDB
`DBDeadlock … Deadlock found when trying to get lock`	1213	Transaction annulée par la certification	Occasionnel : normal. Fréquent : écritures sur plusieurs nœuds
`WSREP has not yet prepared node for application use`	1047	Le nœud atteint est hors quorum ou en transfert	Contrôle de santé HAProxy absent ou trompé
`Too many connections`	1040	Plafond `max_connections` atteint	Trop de workers, fuite de connexions
`Lock wait timeout exceeded`	1205	Une ligne est restée verrouillée trop longtemps	Transaction longue : purge, migration, requête bloquée
`Unknown column …` , `Table … doesn't exist`	1054, 1146	Le schéma ne correspond pas au code	Migration oubliée après une montée de version
`QueuePool limit … overflow … reached, connection timed out`	—	Le pool du processus est épuisé	Service saturé : la base, elle, répond peut-être très bien

Côté MariaDB, le journal d'erreurs (`/var/log/mysql/` , ou `/var/log/kolla/mariadb/`) trace les changements de vue du cluster : lignes `WSREP: view` , `New cluster view` , `Shifting … → SYNCED` .

Indice décisif : une erreur de base touche **tous** les services au même instant. Si un seul service est en erreur, chercher d'abord chez lui — pool, compte, schéma.

 Sunbeam : MySQL InnoDB Cluster, slide 5

Perte de quorum : réamorcer le cluster




```
# /var/lib/mysql/grastate.dat
# GALERA saved state
version: 2.1
uuid:    7f3c2a91-5c1e-11ef-9a41-b7d0c5a8e2f1
seqno:   48211937
safe_to_bootstrap: 0
```

```
# Avec Kolla-Ansible : toute la procédure en une commande
kolla-ansible mariadb-recovery -i inventaire
```

- **Pourquoi le plus avancé :** amorcer depuis un nœud en retard ferait perdre les transactions que seul un autre avait reçues. Les autres nœuds s'aligneraient sur lui.
- **seqno = −1** signifie que le nœud ne s'est pas arrêté proprement : la position réelle se retrouve en rejouant le journal InnoDB.
- **Ne jamais amorcer deux nœuds :** on obtiendrait deux clusters indépendants, qui divergent derrière la même VIP.
- Cas plus simple — nœuds vivants mais tous « non-Primary » : sur le plus avancé, `SET GLOBAL wsrep_provider_options='pc.bootstrap=YES'`.

> Pendant toute l'opération, les API sont indisponibles et les instances continuent de fonctionner. Rien ne presse au point de sauter l'étape 2.

 Sunbeam : action create-backup, slide 5

Hygiène : purger et sauvegarder

La plupart des services ne suppriment pas les lignes : ils les marquent (`deleted` , `deleted_at`). Sans traitement périodique, les tables grossissent, les requêtes ralentissent et les transferts d'état Galera s'allongent.

Service	Commande de purge	Effet
Nova	`nova-manage db archive_deleted_rows --until-complete --all-cells`	Déplace les lignes supprimées vers des tables « shadow »
Nova	`nova-manage db purge --before <date> --all-cells`	Efface définitivement les lignes archivées
Cinder	`cinder-manage db purge <jours>`	Efface les lignes supprimées depuis plus de N jours
Glance	`glance-manage db purge --age_in_days <jours>`	Idem pour les images
Heat	`heat-manage purge_deleted -g days <jours>`	Efface les piles supprimées et leurs événements
Keystone	`keystone-manage trust_flush`	Efface les délégations expirées

Sauvegarder

- **mariabackup** : copie physique à chaud, complète ou incrémentale ; avec Kolla-Ansible : `kolla-ansible mariadb-backup -i inventaire` .
- **mariadb-dump** : export logique, schéma par schéma ; lent sur les gros volumes mais lisible et portable.
- Sauvegarder sur **un** nœud, stocker **hors** du cluster, et tester la restauration.

Trois copies Galera ne sont pas une sauvegarde : une suppression par erreur ou une migration ratée est répliquée instantanément sur les trois nœuds. Une sauvegarde est indispensable **avant chaque montée de version** — c'est la seule voie de retour arrière.

Métriques à surveiller

Métrique	Valeur saine	Signal d'alerte	Ce que cela révèle
Taille du cluster	3	2 : alerte ; 1 : critique	Redondance entamée ; prochaine panne = arrêt des API
État du cluster, par nœud	Primary, Synced	Tout autre état durable	Nœud hors quorum ou bloqué en transfert
Pause du contrôle de flux	Proche de 0	Au-dessus de quelques pour cent	Un nœud trop lent freine toutes les écritures
File de réception locale	Proche de 0	Croissance sur un nœud	Disque ou CPU de ce nœud en cause
Échecs de certification	Croissance lente	Accélération	Écritures concurrentes sur plusieurs nœuds
Connexions / plafond	Sous 70 %	Au-dessus de 85 %	Trop de workers ; refus de connexion imminent
Requêtes lentes	Rares	En hausse	Tables à purger, index manquant, requête non paginée
Latence d'écriture disque	Quelques millisecondes	Dizaines de millisecondes	Stockage inadapté ; première cause de lenteur des API
Taille des schémas	Stable ou croissance douce	Croissance continue	Purge absente
Âge de la dernière sauvegarde	Moins de 24 h	Au-delà	Tâche de sauvegarde en échec

Sources : exporteur Prometheus pour MariaDB (variables d'état et `wsrep_*`), journal d'erreurs, journal des requêtes lentes, statistiques HAProxy pour savoir quel nœud est actif.

L'alerte la plus utile : « `wsrep_cluster_size` inférieur à 3 ». Un cluster à deux nœuds fonctionne parfaitement — et n'a plus aucune marge.

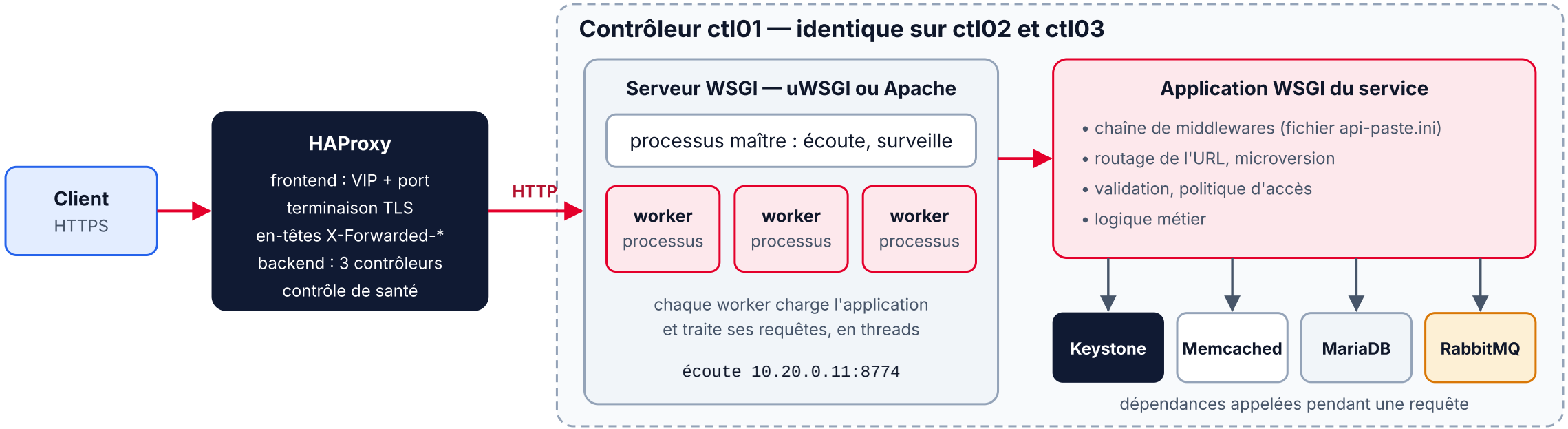
03

Le serveur d'API

C'est la seule partie du cloud que les utilisateurs touchent directement. Deux étages la composent : le répartiteur HAProxy, puis un serveur WSGI qui exécute le code Python du service.

- La pile : HAProxy, serveur WSGI, application
- Middlewares, sockets, surface exposée
- Tests manuels et codes HTTP
- Modes d'hébergement et workers
- Délais en cascade
- Journaux, état des backends, saturation

La pile : du répartiteur au code du service



HAProxy

Reçoit toutes les connexions, déchiffre, choisit un contrôleur sain. Ne connaît rien d'OpenStack : il ne voit que du HTTP.

Serveur WSGI

Fait tourner le code Python : gère les processus, les threads, les redémarrages. C'est lui qui fixe la capacité de traitement.

Application

Le service lui-même. Une requête n'aboutit que si **toutes** ses dépendances répondent : la disponibilité de l'API est le produit des leurs.

Modes d'hébergement de l'application

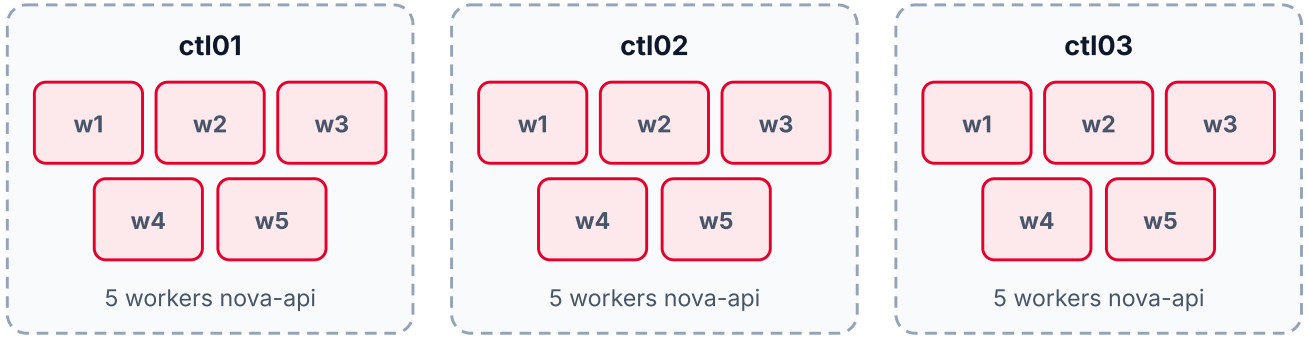
Mode	Principe	Réglage de la capacité	Situation
uWSGI	Serveur d'applications dédié : un maître, des workers, chacun avec ses threads	`processes`, `threads` dans le fichier `.ini` du service	Mode de référence des déploiements récents
Apache + mod_wsgi	Apache héberge l'application dans des processus démons	`WSGIDaemonProcess … processes=N threads=M`	Très répandu ; apporte les journaux d'accès d'Apache
Serveur intégré (eventlet)	Le service écoute lui-même, avec des fils coopératifs	Option `*_workers` du service	Historique ; en cours de retrait dans tout OpenStack

Service	Module WSGI (versions récentes)
Nova, API	`nova.wsgi.osapi_compute`
Nova, métadonnées	`nova.wsgi.metadata`
Keystone	`keystone.wsgi.api`
Neutron	`neutron.wsgi.api`
Cinder	`cinder.wsgi.api`
Placement	`placement.wsgi.api`

- **Nova** a supprimé ses scripts de serveur eventlet (`nova-api` , `nova-api-metadata`) : depuis sa version 33.0.0, seul le déploiement WSGI est accepté.
- L'outil de déploiement génère le fichier de configuration du serveur WSGI et y inscrit le module à charger.
- Savoir quel mode est en place : `ps aux | grep -E 'uwsgi|httpd|apache2'` sur un contrôleur.

> Le mode ne change rien pour les utilisateurs ni pour HAProxy. Il change **où** se règle le nombre de workers et **où** s'écrivent les journaux d'accès.

Workers : la capacité, et ce qu'elle coûte en aval



Ce que ces 15 processus peuvent ouvrir

3 contrôleurs × **5** workers = **15** processus nova-api

× jusqu'à **55** connexions SQL chacun (5 + 50) = **825 connexions possibles**

× plusieurs connexions RabbitMQ chacun

× quelques centaines de Mo de mémoire chacun

Et autant pour chaque autre API : Neutron, Cinder, Keystone, Glance, Placement, Heat…

- Un **worker** traite une requête à la fois par thread. Trop peu de workers : les requêtes attendent, HAProxy renvoie des 504.
- Trop de workers : mémoire épuisée sur les contrôleurs et plafond de connexions de la base dépassé.
- La bonne valeur se **mesure** : temps de réponse, file d'attente HAProxy, `Threads_connected`.

Où se règle le nombre de workers
uWSGI : `processes` et `threads`
mod_wsgi : directive `WSGIDaemonProcess`
Kolla-Ansible : variable `openstack_service_workers`

Piège du défaut « un worker par cœur » : sur un contrôleur de 64 cœurs, dix services à 64 workers font 640 processus et des dizaines de milliers de connexions potentielles. Toujours borner.

Les middlewares : ce que chaque requête traverse

Middleware	Fonction	Réglage associé
cors	Autorise les appels depuis un navigateur sur un autre domaine	Section `[cors]`, `allowed_origin`
http_proxy_to_wsgi	Rétablit protocole et hôte d'origine à partir des en-têtes posés par HAProxy	`[oslo_middleware]` `enable_proxy_headers_parsing`
request_id	Attribue l'identifiant `req-…`	—
healthcheck	Répond sur `/healthcheck` sans authentification	Section `[healthcheck]`
sizelimit	Refuse les corps trop volumineux	`max_request_body_size` (114 688 octets)
authtoken	Valide le jeton, renseigne l'identité	Section `[keystone_authtoken]`

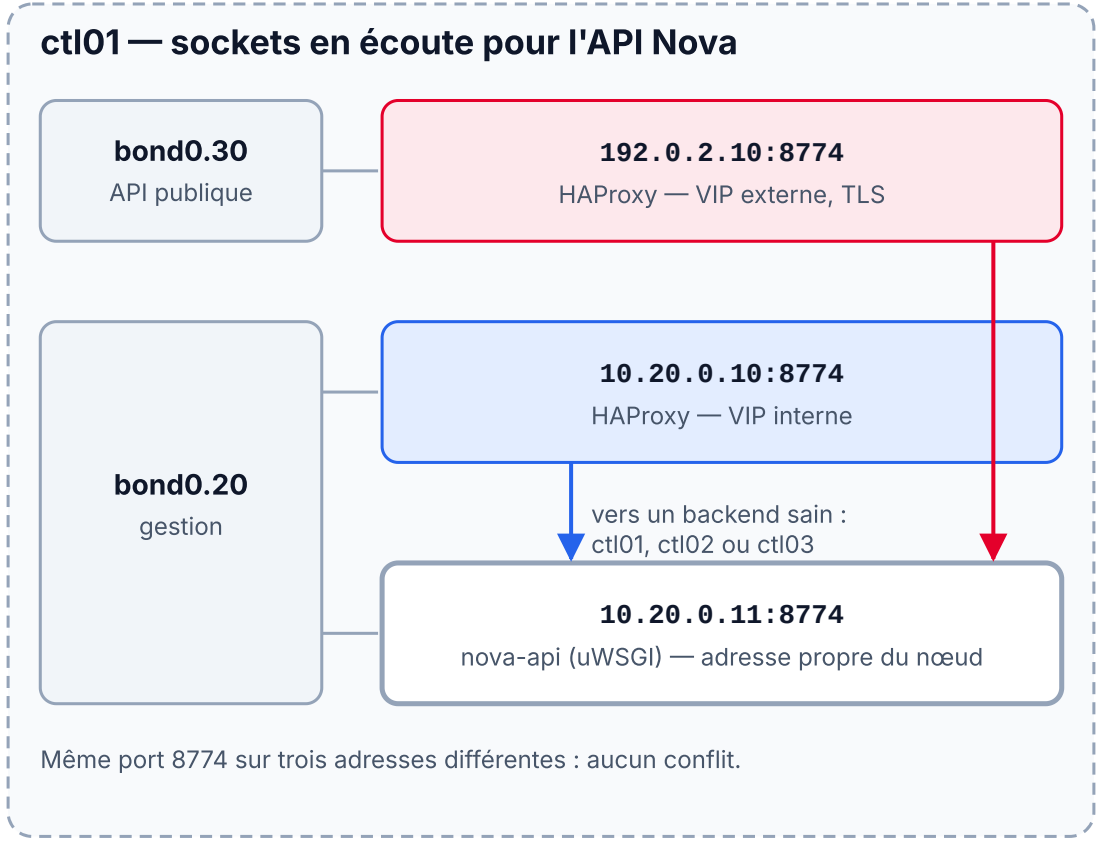
Sans `enable_proxy_headers_parsing`, le service se croit appelé en HTTP sur son adresse interne : les liens qu'il renvoie sont injoignables pour le client.

```
# /etc/nova/api-paste.ini (extrait)
[composite:openstack_compute_api_v21]
use = call:nova.api.auth:pipeline_factory_v21
keystone = cors http_proxy_to_wsgi compute_req_id
    faultwrap request_log sizelimit osprofiler
    authtoken keystonecontext osapi_compute_app_v21
```

```
# nova.conf — ce dont authtoken a besoin
[keystone_authtoken]
www_authenticate_uri = https://cloud.example.org:5000
auth_url = https://int.cloud.lan:5000
memcached_servers = 10.20.0.11:11211,10.20.0.12:11211,…
auth_type = password
username = nova
project_name = service
service_token_roles_required = true
```

- L'ordre compte : une requête sans jeton valide est rejetée par **authtoken** avant d'atteindre le code du service.
- `memcached_servers` évite d'interroger Keystone à chaque requête : sans cache, Keystone devient le goulot de tout le cloud.

Exposition : qui écoute sur quelle adresse




```
# haproxy.cfg — API Nova (forme type)
frontend nova_api_external_front
    mode http
    bind 192.0.2.10:8774 ssl crt /etc/haproxy/certs/cloud.pem
    http-request set-header X-Forwarded-Proto https
    option forwardfor
    default_backend nova_api_back

frontend nova_api_internal_front
    mode http
    bind 10.20.0.10:8774
    default_backend nova_api_back

backend nova_api_back
    mode http
    option httpchk
    server ctl01 10.20.0.11:8774 check inter 2000 rise 2 fall 5
    server ctl02 10.20.0.12:8774 check inter 2000 rise 2 fall 5
    server ctl03 10.20.0.13:8774 check inter 2000 rise 2 fall 5
```

- L'application n'écoute **jamais** sur l'interface publique : seul HAProxy s'y présente.
- `rise 2 fall 5` : un backend est retiré après 5 contrôles ratés, réintégré après 2 réussis.
- Vérifier : `ss -ltnp | grep 8774` doit montrer exactement ces adresses, pas `0.0.0.0`.

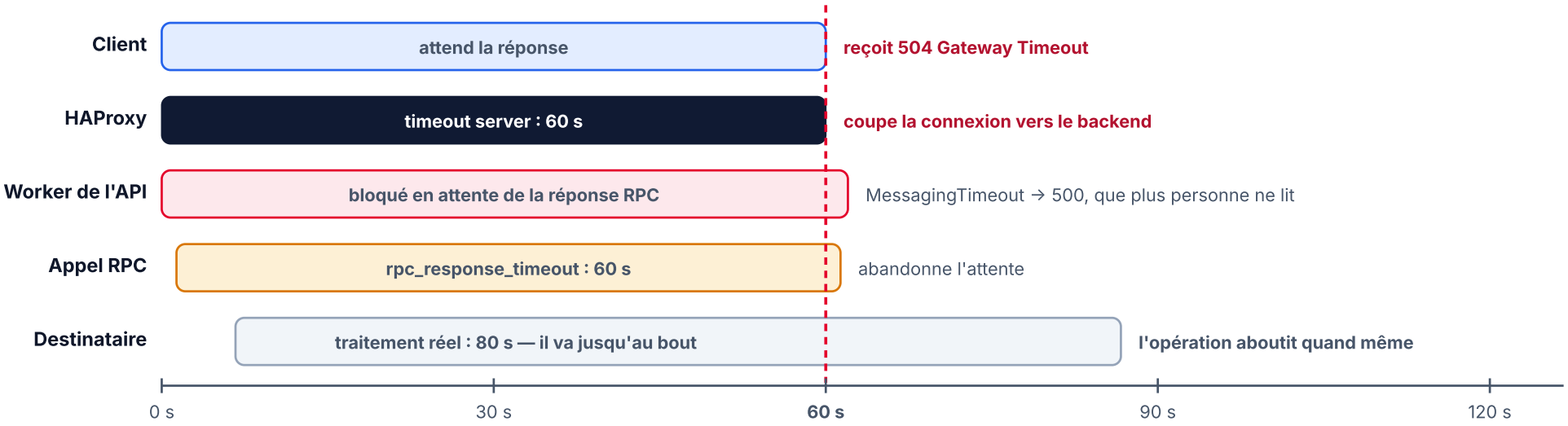
Exposition : la surface à maîtriser

Élément exposé	Ce qu'il permet sans précaution	Où le maîtriser
Découverte des versions (`GET /`)	Accessible sans jeton : révèle versions et microversions prises en charge	Comportement normal ; ne rien y ajouter
Contrôle de santé (`/healthcheck`)	Accessible sans jeton ; le mode détaillé expose des informations internes	Mode détaillé désactivé ; filtrer le chemin sur la VIP externe
Taille des requêtes	Corps énormes qui saturent la mémoire des workers	`max_request_body_size` ; limites HAProxy
Listes de ressources	Requêtes très coûteuses pour la base	Pagination imposée : `[api] max_limit` (1000 dans Nova)
Débit d'appels	Aucune limitation dans la plupart des services : un client peut saturer les workers	HAProxy (tables de suivi), pare-feu applicatif
Chiffrement	Protocoles et suites faibles s'ils restent activés	Directive `bind … ssl` de HAProxy : versions TLS, suites, HSTS
Origines autorisées (CORS)	Appels depuis n'importe quel site web	Section `[cors]` : liste explicite
Messages d'erreur	Traces Python renvoyées au client	`debug = false` en production
Consoles (port 6080) et Horizon	Autres portes d'entrée publiques, souvent oubliées	Mêmes exigences TLS que les API
Interface interne	Atteignable depuis une instance si le filtrage est défaillant	Segmentation réseau ; TLS interne ; jetons de service

Tout ce qui est public passe par HAProxy : c'est là que se placent le chiffrement, les limites et le filtrage. L'application, derrière, suppose qu'un tri a déjà été fait.

Vérifier depuis l'extérieur : un balayage de ports de la VIP externe ne doit montrer que les ports des API publiées, Horizon et les consoles — ni 3306, ni 5672, ni 15672, ni 11211.

Délais en cascade : pourquoi un 504 n'annule rien



Étage	Délai	Réglé où
Client	Propre à l'outil	Option `--timeout` de la CLI, SDK
HAProxy	`timeout client` , `timeout server`	`haproxy.cfg`
Serveur WSGI	Durée maximale d'une requête	uWSGI (`harakiri`), Apache
RPC	`rpc_response_timeout`	Fichier du service
Base	`pool_timeout` , verrous	Service et MariaDB

- Le **premier** délai qui expire décide de ce que voit le client ; les étages suivants continuent de travailler.
- Un **504** signifie « pas de réponse à temps », pas « opération échouée ». Relancer aveuglément crée des doublons.
- Après un 504, **vérifier l'état** de la ressource avant toute nouvelle tentative.
- Allonger les délais repousse le symptôme sans traiter la lenteur. La bonne question : pourquoi 80 s ?

> C'est aussi pourquoi les opérations longues sont des « cast » : l'API répond 202 tout de suite et n'attend rien.

Diagnostic : interroger l'API à la main

```
# 1. La VIP répond-elle ? (aucun jeton requis)
curl -s https://cloud.example.org:8774/ | python3 -m json.tool

# 2. Chaque backend, directement, depuis un contrôleur
for h in 10.20.0.11 10.20.0.12 10.20.0.13; do
  curl -s -o /dev/null \
       -w "$h  %{http_code}  %{time_total}s\n" http://$h:8774/
done

# 3. Contrôle de santé (si le middleware est activé)
curl -i http://10.20.0.11:8774/healthcheck

# 4. Un appel authentifié, tous les échanges affichés
openstack --debug --os-interface internal server list

# 5. Où passe le temps ?
curl -s -o /dev/null -w "tcp %{time_connect}  tls %{time_appconnect}
  premier octet %{time_starttransfer}  total %{time_total}\n" \
  https://cloud.example.org:5000/v3
```

Constat	Conclusion
La VIP échoue, les trois backends répondent	HAProxy ou Keepalived : VIP absente, certificat, frontend
Un backend sur trois échoue	Serveur WSGI de ce contrôleur ; HAProxy doit déjà l'avoir retiré
Tous répondent à `/`, l'appel authentifié échoue	Keystone, Memcached, ou la base derrière le service
Interface interne correcte, publique en échec	Réseau ou TLS côté externe ; catalogue erroné
Premier octet tardif, connexion rapide	Le temps est passé dans le service ou ses dépendances
Connexion TCP lente	Réseau, ou file d'attente de connexions saturée

Démarche : aller du plus simple au plus complet. `GET /` ne dépend ni de Keystone ni de la base : s'il répond, HAProxy et le serveur WSGI sont hors de cause, et le problème est en aval.

`--debug` affiche chaque requête, chaque code de retour et l'identifiant `req-…` à rechercher ensuite dans les journaux.

Diagnostic : ce que dit le code HTTP

Code	Émis par	Signification	Où regarder
400	Application	Requête mal formée ou paramètre invalide	Corps de la réponse : le message est explicite
401	Middleware authtoken	Jeton absent, expiré ou non validé	Keystone, Memcached, horloges, clés Fernet
403	Application (politique)	Jeton valide mais rôle insuffisant, ou quota atteint	Rôles de l'utilisateur, fichier de politique, quotas
404	Application	Ressource inconnue — ou appartenant à un autre projet	Projet du jeton ; URL du catalogue
406, 409	Application	Microversion refusée ; état de la ressource incompatible avec l'action	Version demandée ; `status` et `task_state`
500	Application	Exception non prévue pendant le traitement	Journal de l'API : la trace nomme la base, le bus ou un autre service
502	HAProxy	Le backend a fermé la connexion ou répondu de façon invalide	Worker tué ou redémarré : journal du serveur WSGI
503	HAProxy ou authtoken	Aucun backend sain ; ou Keystone injoignable par le service	État des backends ; journal de l'API
504	HAProxy	Le backend n'a pas répondu dans le délai	Lenteur en aval : base, bus, workers saturés

4xx : le client

La requête a été comprise, le service fonctionne : revoir la demande, l'identité ou les droits.

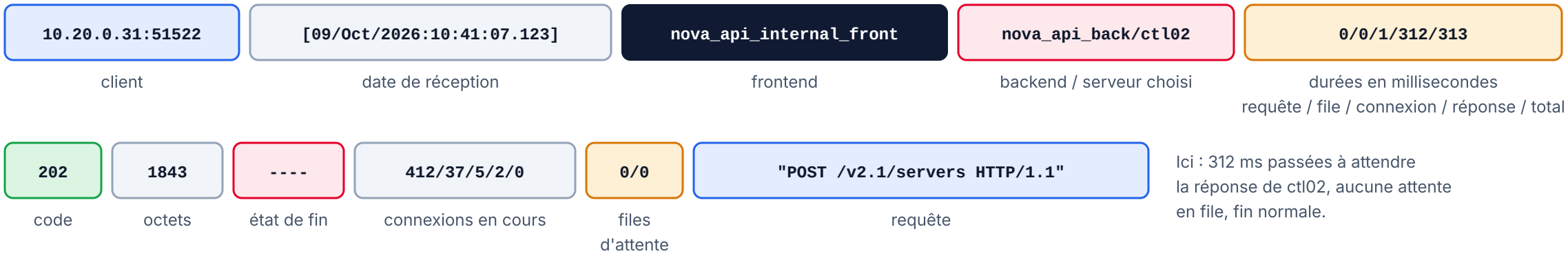
500 : le service

Le code s'est exécuté et a échoué. La réponse est dans le journal du service, à l'identifiant de la requête.

502, 503, 504 : l'infrastructure

La requête n'a pas été traitée normalement. Le journal du service peut être vide : commencer par HAProxy.

Journaux : lire une ligne de HAProxy



État de fin	Lecture	Code associé
`----`	Échange terminé normalement	Celui de l'application
`sH--`	Délai dépassé en attendant la réponse du serveur	504
`SC--`	Connexion refusée par le serveur	503
`SH--`, `PH--`	Le serveur a coupé ou répondu de façon invalide	502
`cD--`, `CD--`	Le client a abandonné ou expiré pendant le transfert	—
`<NOSRV>` **à la place du serveur**	Aucun backend disponible	503

- **Quatrième durée élevée** : le backend a été lent → chercher dans le journal du service.
- **Deuxième durée élevée**, ou files non nulles : tous les workers sont occupés → saturation.
- **Troisième durée élevée** : établissement de connexion lent → réseau ou backend débordé.

> Le journal du service (`nova-api.log`) donne le détail **à l'intérieur** du traitement, avec l'identifiant `req-…` ; celui de HAProxy donne la vue **de l'extérieur**, y compris pour les requêtes qui n'ont jamais atteint le service.

Diagnostic : l'état des backends vu par HAProxy

```
# Interroger HAProxy par son socket d'administration
echo "show stat" | socat stdio /run/haproxy/admin.sock \
  | cut -d, -f1,2,3,5,18,22,24 | grep nova_api

# pxname, svname, qcur, scur, status, chkfail, lastchg
nova_api_back,ctl01,0,12,UP,0,86211
nova_api_back,ctl02,0,11,UP,0,86209
nova_api_back,ctl03,0,0,DOWN,7,312
nova_api_back,BACKEND,0,23,UP,,86211
```

```
# Retirer un contrôleur avant maintenance, puis le remettre
echo "set server nova_api_back/ctl03 state drain" \
  | socat stdio /run/haproxy/admin.sock
echo "set server nova_api_back/ctl03 state ready" \
  | socat stdio /run/haproxy/admin.sock
```

> Le chemin du socket et la présence d'une page de statistiques HTTP dépendent de l'outil de déploiement.

Champ	Sens	À surveiller
`status`	UP, DOWN, DRAIN, MAINT	Tout serveur DOWN ; un backend entier DOWN = 503
`scur`	Sessions en cours	Répartition très inégale entre serveurs
`qcur`	Requêtes en file d'attente	Toute valeur durablement non nulle
`chkfail`	Contrôles de santé ratés	Compteur qui augmente
`lastchg`	Secondes depuis le dernier changement d'état	Valeur faible : bascule récente ou instable
`hrsp_5xx`	Réponses 5xx cumulées	Hausse sur un seul serveur

- Un serveur qui **oscille** entre UP et DOWN indique un backend à la limite : workers saturés ou dépendance lente.
- Le contrôle de santé par défaut ne vérifie que la réponse HTTP : un service qui répond à `/` mais dont la base est en panne reste « UP ».
- **Drainer** un serveur avant de l'arrêter : il termine ses requêtes en cours et n'en reçoit plus de nouvelles.

Saturation : la reconnaître et en trouver la cause

Signe	Mesure	Cause probable	Action
Latence qui monte sur toutes les requêtes	Durées dans le journal HAProxy	Workers tous occupés, ou dépendance lente	Distinguer : file HAProxy non nulle, ou temps de réponse du backend élevé
504 par vagues	État de fin `sH--`	Requêtes plus longues que le délai du répartiteur	Trouver la requête lente ; base, bus
503 intermittents	Serveurs qui oscillent UP / DOWN	Contrôles de santé qui expirent sous la charge	Capacité des workers ; mémoire du contrôleur
502 isolés	Journal du serveur WSGI	Worker tué : mémoire épuisée, durée maximale atteinte	Mémoire par worker ; fuite ; requête anormale
CPU des contrôleurs à 100 %	Par processus	Trop de workers pour le nombre de cœurs, ou appels en rafale	Borner les workers ; identifier le client bavard
Erreurs de pool SQL	« QueuePool limit … reached »	Plus de requêtes simultanées que de connexions	Requêtes lentes d'abord, taille du pool ensuite
Keystone lent, tout ralentit	Durée des validations de jeton	Cache Memcached absent ou inaccessible	`memcached_servers` dans chaque service

Identifier le client à l'origine de la charge

- Journal HAProxy : adresses sources et URL les plus fréquentes.
- Journal de l'API : projet et utilisateur de chaque requête.
- Coupables classiques : supervision trop fréquente, script en boucle, outil d'infrastructure as code qui rafraîchit des milliers de ressources.

Voir ce que fait un processus bloqué : les services savent produire un rapport d'état de leurs fils d'exécution (« Guru Meditation Report », bibliothèque oslo.reports). Il montre où chaque thread est arrêté — attente SQL, attente RPC, verrou.

MÉTHODE

Un arbre de diagnostic commun aux trois composants

Un appel API échoue ou n'aboutit pas

Serveur d'API

Signes

- 502, 503, 504, connexion refusée
- 401 par intermittence
- latence en hausse sur toutes les requêtes

Vérifier, dans l'ordre

1. curl sur la VIP, puis sur chaque backend
2. état des backends dans HAProxy
3. journal HAProxy : durées, état de fin
4. journal de l'API à l'identifiant req-…
5. Keystone, Memcached, horloges

Reprise type : retirer le backend fautif

File de messages

Signes

- 202 reçu, mais ressource figée (BUILD…)
- MessagingTimeout dans un journal
- services « down » en série

Vérifier, dans l'ordre

1. cluster_status, alarmes
2. list_queues : messages, consommateurs
3. journal du service destinataire
4. connexions : état, nombre
5. réseau entre le nœud et les contrôleurs

Reprise type : rétablir le consommateur

Base de données

Signes

- 500 avec DBConnectionError, DBDeadlock
- toutes les API en erreur en même temps
- erreur 1047 : nœud hors quorum

Vérifier, dans l'ordre

1. wsrep_cluster_size, wsrep_cluster_status
2. nœud actif vu par HAProxy
3. Threads_connected / max_connections
4. requêtes lentes, verrous
5. latence disque des contrôleurs

Reprise type : rétablir le quorum

Dans tous les cas : partir de l'identifiant de requête et suivre l'opération dans les journaux, étage par étage.

SYNTHÈSE

Fiche réflexe

	File de messages	Base de données	Serveur d'API
Rôle en une phrase	Porte les ordres entre processus d'un service	Garde l'état de référence du cloud	Reçoit et traite les requêtes REST
Ports	5672 · 4369, 25672 · 15672, 15692	3306 · 4567, 4568, 4444	Un par API, sur VIP externe et VIP interne
Accès des services	Direct, liste des 3 nœuds	Par la VIP interne, 1 nœud actif	Par la VIP, 3 backends actifs
Option clé côté service	`transport_url`	`[database] connection`	`[keystone_authtoken]`, catalogue
Première commande	`rabbitmqctl cluster_status`	`SHOW STATUS LIKE 'wsrep_%'`	`curl` VIP et backends
Deuxième commande	`rabbitmqctl list_queues name messages consumers`	`SHOW PROCESSLIST`	`show stat` sur le socket HAProxy
Message caractéristique	`MessagingTimeout`	`DBConnectionError`, `DBDeadlock`	502, 503, 504
Tolère la perte de	1 nœud sur 3	1 nœud sur 3	2 backends sur 3
Panne majeure	Partition, alarme mémoire	Perte de quorum	Aucun backend sain, VIP absente
Reprise	Rétablir la majorité, redémarrer les services déconnectés	Réamorcer depuis le nœud le plus avancé	Rétablir un backend ; vérifier Keepalived
À ne jamais faire	Purger une file RPC en production	Amorcer deux nœuds ; modifier le schéma à la main	Exposer l'application sans HAProxy

Trois signatures : le bus tombe, les opérations se figent sans erreur ; la base tombe, toutes les API répondent en erreur ; une API tombe, un seul service est touché. Dans les trois cas, les instances continuent de tourner.